

Bridging the Sim-to-Real Gap with Domain Randomization

Chiara Roberta Casale
Politecnico di Torino

s322574@studenti.polito.it

Yegane Bagheri
Politecnico di Torino

s327779@studenti.polito.it

Abstract

One of the main challenges in applying Reinforcement Learning (RL) in robotics is the performance gap between simulation and the real world. In this project, we analyzed various RL algorithms and highlighted how the sim-to-real gap affects the different results. We then explored possible solutions to this problem, particularly using the Domain Randomization paradigm, which tries to expose the agent to as many different scenarios as possible to cover potential variations in the real world.

1. Introduction

This project aims to train a Reinforcement Learning agent with various state-of-the-art algorithms, with the objective of bridging the Sim-to-Real gap. We present the main paradigms at the base of this framework. The code of the presented methods can be found at the project repository [1].

1.1. Reinforcement Learning

Reinforcement learning (RL) is a framework that trains an *agent* through direct interaction with its *environment*. RL differs from the two main machine learning paradigms, supervised and unsupervised learning: it does not use examples of the desired behavior to choose its actions nor tries to find a hidden structure.

On each step, the agent sees a partial description (observation) of the environment's current *state* S_t . Based on the *observation*, the agent decides on which *action* A_t to take. The agent also perceives from the environment a single number called *reward* R_t . The agent's objective is to maximize its cumulative reward, called *return*. The agent decides which actions to take through a rule called *policy* π .

The following are other important concepts to define a RL problem. A sequence of states and actions is called a *trajectory* τ :

$$\tau = (S_0, A_0, S_1, A_1 \dots) \quad (1)$$

Episodes are subsequences of the agent-environment interaction which are naturally divided based on certain events.

The *discounted return* is defined as follows:

$$G_t = R_1 + \gamma R_2 + \gamma^2 R_3 + \dots = \sum_{t=0}^{\infty} \gamma^t R_{t+1} \quad (2)$$

with discount factor $\gamma \in (0, 1]$.

RL aims to select a policy that maximizes the *expected return* $J(\pi)$:

$$\max_{\pi} J(\pi) = \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t R_{t+1} \right] \quad (3)$$

with $T \in (0, +\infty)$.

The *state-value function* $V^{\pi}(s)$ gives the expected return if you start in state s and always act according to policy π :

$$V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi} [G_t(\tau) | S_0 = s] \quad (4)$$

The *action-value function* $Q^{\pi}(s, a)$ gives the expected return if you start in state s , take an arbitrary action a , and then always act according to policy π :

$$Q^{\pi}(s, a) = \mathbb{E}_{\tau \sim \pi} [G_t(\tau) | S_0 = s, A_0 = a] \quad (5)$$

1.2. Sim-to-Real transfer

One of the most well-known applications of reinforcement learning is to train robots to do specific tasks. Training an agent relies on gaining experience, which can have various drawbacks when implemented with robots. A solution can be using the Sim-to-Real transfer [2]. The agent is trained in a simulated environment and then is transferred to a real robot in a real environment. The primary motivation behind Sim-to-Real transfer is the reduction of costs and risks associated with training real robots. Moreover, it can reduce the training time and make data collection easier.

When implementing Sim-to-Real transfer, we face what is called the *reality gap*. It refers to the mismatch between the results we see in the simulated environment and the real world. These discrepancies can arise from various factors,

including differences in sensor noise, and actuation dynamics, and there can be real-world experiences and tasks for which the agent was not trained in the simulation. As a result, a policy that performs well in simulation might not necessarily perform well in the real world.

There are different strategies to address the reality gap. In this project, we focused on Domain Randomization [3]. Domain Randomization involves training the policy on a wide range of simulated environments with randomized parameters. The goal is to expose the agent to enough simulated variability of the parameters at training time so that it learns a robust policy capable of generalizing to the real world. This method assumes that the variations in the simulation cover the possible variations in the real world.

For feasibility reasons, we simulated the Sim-to-Real transfer task using a controlled sim-to-sim scenario. The agent was trained in an environment we called the source environment and then tested in the target environment. To mimic the reality gap, discrepancies were manually added. Specifically, the torso mass of the Hopper robot is shifted by 1 kg between the source and target domains.

1.3. Simulation

The environment used in this project is the Hopper environment from OpenAI Gym. It is designed to simulate a one-legged robot, which consists of a torso, thigh, leg, and foot. The body parts are connected through three hinge joints. The primary objective for the Hopper is to learn to hop forward as rapidly as possible while maintaining stability. Movement is achieved through the application of torques to the joints. The robot is considered alive as long as it remains upright and maintains balance; if it falls or becomes unstable, it's considered dead and the episode ends.

The simulated environment is a flat, planar surface on which the Hopper operates. In the target environment, the robot's body parts have the following masses: 3.53 kg for the torso, 3.93 kg for the thigh, 2.71 kg for the leg, and 5.09 kg for the foot. In the source environment, the only difference is the mass of the torso, which is decreased by 1 kg compared to the target. The state space is continuous and includes information about the robot and its joints in each state (positions, velocities, angles). The action space is also continuous, representing the three torques applied to the robot's joints.

Three different components are considered in the reward function:

- *Alive Bonus*: The robot receives a reward of +1 for each timestep it remains alive.
- *Forward Progress Reward*: The reward is based on the distance the Hopper moves forward in each timestep; the higher the distance, the higher the reward. It's calculated as the difference between its new position and

the older position, divided by the duration of the action.

- *Control Penalty*: This negative reward penalizes excessive actions to encourage smooth and efficient movements, to avoid abrupt accelerations.

2. Related Work

Various papers are cited throughout the paper, but in particular, we want to highlight the following related works:

- *Domain Randomization for transferring deep neural networks from simulation to the real world* by Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017) [4] is one of the first papers on Domain Randomization. Domain Randomization is explained in sections 1.2 and 3.4 to understand our project, but the paper goes into deeper details about how the algorithm works, also with an application that it's different from ours. Domain Randomization is not the only state-of-the-art algorithm one can use to overcome the Sim-to-Real gap, but it's the one we focused on in our work.
- *Solving Rubik's cube with a robot hand* by OpenAI (2019) [5] is the paper proposing the algorithm Automatic Domain Randomization (explained in section 3.4). Starting from the first papers like the aforementioned work by Tobin et al., new and more complex ideas of Domain Randomization have been experimented with, and Automatic Domain Randomization is one of them.

3. Methodology

We focused our attention on three algorithms in particular: REINFORCE, Actor-Critic, and Proximal Policy Optimization. These algorithms represent a progression from basic to more advanced techniques. They're implementations of the policy gradient method, which relies on gradient ascent to optimize the policy. The policy gradient theorem provides a way to compute the gradient of the expected return with respect to the policy parameters θ :

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\pi}[Q^{\pi}(s, a) \nabla_{\theta} \ln \pi_{\theta}(a|s)] \quad (6)$$

3.1. REINFORCE

REINFORCE relies on computing the discounted return G_t and uses the policy gradient theorem for updating the policy parameters. It works because the expectation of the gradient is equal to the actual gradient:

$$\begin{aligned} \nabla_{\theta} J(\theta) &= \mathbb{E}_{\pi}[Q^{\pi}(s, a) \nabla_{\theta} \ln \pi_{\theta}(a|s)] \\ &= \mathbb{E}_{\pi}[G_t \nabla_{\theta} \ln \pi_{\theta}(A_t|S_t)] \end{aligned} \quad (7)$$

Algorithm 1 REINFORCE (Vanilla Policy Gradient)

Algorithm parameter: step size $\alpha > 0$
Initialize: policy parameter θ
loop forever (for each episode)
 Generate one episode following policy π_θ :
 $S_0, A_0, R_1 \dots, S_{T-1}, A_{T-1}, R_T$
 for $t = 0, 1, \dots, T-1$ **do**
 $G_t \leftarrow \sum_{k=t+1}^T \gamma^{k-t-1} R_k$
 $\theta \leftarrow \theta + \alpha \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t|S_t)$
 end for
end loop

This equation holds true because $Q^\pi(S_t, A_t) = \mathbb{E}_\pi[G_t|S_t, A_t]$, as we’ve seen with equation 5.

The process behind the REINFORCE algorithm is explained in detail in Algorithm 1 [6].

A possible variation of REINFORCE is to subtract a baseline value from the return G_t . With this change, the variance of gradient estimation is reduced while keeping the bias unchanged.

3.2. Actor-Critic

Algorithm 2 Actor-Critic Algorithm

Algorithm parameters: Step sizes $\alpha_\theta > 0, \alpha_w > 0$
Initialize: policy parameter θ and state-value weights w
loop forever (for each episode)
 Initialize: S_0 (first state of episode)
 $I \leftarrow 1$
 while S_t is not terminal (for each time step t) **do**
 $A_t \sim \pi_\theta(\cdot|S_t)$
 Take action A_t , observe S_{t+1}, R_t
 if S_{t+1} is terminal **then**
 $V_w(S_{t+1}) \leftarrow 0$
 end if
 $\delta_t \leftarrow R_t + \gamma V_w(S_{t+1}) - V_w(S_t)$
 $w \leftarrow w + \alpha_w \delta_t \nabla V_w(S_t)$
 $\theta \leftarrow \theta + \alpha_\theta I \delta_t \nabla \ln \pi_\theta(A_t|S_t)$
 $I \leftarrow \gamma I$
 $S_t \leftarrow S_{t+1}$
 end while
end loop

The Actor-Critic method combines elements of policy-based and value-based approaches. It addresses some limitations of REINFORCE by using a value function to reduce variance in the policy gradient estimates. The method involves an actor (policy) and a critic (value function).

The critic updates the parameters w of the value function (in our case a state-value function $V_w(s)$). The actor updates the policy parameters θ , in the direction suggested by the critic.

The policy update is guided by the temporal difference (TD) error δ_t , which provides a more accurate and lower variance estimate of the return. The actor’s policy is updated using the gradient of the expected return, modulated by the TD error. The critic updates its value function parameters to minimize the TD error, using a mean squared error loss.

The process behind the REINFORCE algorithm is explained in detail in Algorithm 2 [6].

3.3. Proximal Policy Optimization

Proximal Policy Optimization (PPO) [7] is a state-of-the-art policy gradient method that optimizes the policy using the concept of Trust Region. If the policy is performing well, the training procedure should perform the updates in such a way that the new policy has a behavior similar to the old one. To do so, PPO uses a clipped objective function to limit the deviation of the new policy from the old policy, ensuring that updates are not drastic, but instead small and controlled.

Compared to simpler methods like REINFORCE and Actor-Critic, it yields higher rewards with more stable training. For this reason, moving forward, we’ll focus on this algorithm and discard the others.

3.4. Domain Randomization

As explained in Section 1.2, Domain Randomization is one of the possible algorithms to bridge the Sim-to-Real gap. It does so by exposing the agent to randomized parameters to cover the potential variations in the real world. In particular, we implemented Uniform Domain Randomization (UDR), which is how Domain Randomization was originally implemented [4], and Automatic Domain Randomization (AutoDR) [5]. In UDR, each randomization parameter is bounded by an interval and each parameter is sampled within the range in a uniform distribution. AutoDR takes the idea of UDR, sampling the environment parameters in a *fixed* uniform distribution, and expands on it. The main difference is that in AutoDR, the uniform distribution’s bounds *change* dynamically throughout the training.

At each iteration, the AutoDR algorithm randomly selects one of the environment parameters and fixes it to its lower or upper bound (*boundary sampling*). The remaining parameters are then sampled uniformly within their current bounds.

The episode reward is then computed and appended to a buffer associated with the sampled boundary of the selected parameter. Once enough performance data is collected, it is averaged and compared to thresholds.

If the average model performance is better than the upper threshold, the selected parameter’s range is expanded: if the sampled boundary is the upper bound, it increases; if it is the lower bound, it decreases. Conversely, if the aver-

age performance is below a lower threshold, the parameter’s range is contracted: if the sampled boundary is the upper bound, it decreases; if it is the lower bound, it increases.

4. Experiments

4.1. REINFORCE vs Actor-Critic

We trained three algorithms:

- REINFORCE
- REINFORCE with constant baseline $b = 20$.
- Actor-Critic

and compared their performance. To do so, we trained each algorithm for 20 000 episodes in the source environment. For every 250 training episodes, we tested the agent for 10 episodes in the source environment. To address the high variance typical of reinforcement learning, we trained 10 different policies for every algorithm and then grouped the plots by their mean.

4.2. Proximal Policy Optimization

To implement PPO, we used the `stable-baseline3` [8] library.

First, we implemented a grid search to find the best hyperparameter. This was done through the Optuna [9] library, which is the standard procedure when implementing stable-baseline3. Optuna is a framework for hyperparameter optimization that can easily explore large search spaces and prune unpromising evaluations (or trials). In particular, we implemented 100 trials, where the agent was trained in the target environment for 100 000 episodes in each trial. The agent is tested in the target environment every 10 000 episodes, to evaluate the return and see if the trial should be pruned. The testing lasts for 50 episodes.

As a pruner, we periodically compared the agent’s performance to its previous evaluations to ensure that the return was consistently improving (within a threshold). This approach allowed us to identify and stop unpromising trials early. Moreover, we achieved results where the performance generally improved steadily even after 100 000 episodes. Without this pruning, we oftentimes observed high returns at the final evaluation followed by a decline in performance.

The values used in the hyperparameter tuning are listed in Table 1. The best combination of hyperparameters is the one highlighted in bold.

With the best hyperparameters, we trained the agent in three different scenarios:

- trained in the source environment and tested in the source environment ($Source \rightarrow Source$)

Hyperparameters	Search Space
Batch size	{32, 64 , 128, 256, 512}
Steps	{64, 128, 256, 512, 1024, 2048 }
Gamma	{ 0.99 , 0.995, 0.999}
Learning rate	{1e-2, 3e-3, 5e-3, 1e-3, 3e-4 , 5e-4}

Table 1. Search Space for PPO

Body Part	Mass (in kg)
Thigh	3.92699082
Leg	2.71433605
Foot	5.0893801

Table 2. Masses of the Hopper’s randomized body parts

Hyperparameters	Search Space
Range	{0.25, 0.5 , 0.75, 1.0, 1.25, 1.5, 1.75, 2.0, 2.25, 2.5}
Scaling	{0.75, 1 , 1.5, 2}

Table 3. Search Space for UDR

- trained in the source environment and tested in the target environment ($Source \rightarrow Target$)
- trained in the target environment and tested in the target environment ($Target \rightarrow Target$)

For every scenario, the training lasted 1 000 000 episodes. At the end of the training, we tested the three agents for 50 episodes.

4.3. Uniform Domain Randomization

To implement Uniform Domain Randomization, we varied the masses of the Hopper’s body parts, except for the torso remaining fixed at a 1 kg difference between source and target. The agent encounters different masses sampled from a uniform distribution at each training episode.

Our starting point was the default values of the masses, listed in Table 2. We changed these values, first through scaling. Then we subtracted and summed a range to find the lower and upper bound respectively. We used the lower and upper bounds to compute the uniform distribution.

To find the best values for range and scaling, we used a GridSearch through the Optuna library. In particular, we evaluated the hyperparameters by training the agent in the source environment and testing it in the source environment. As a pruner, we used MedianPruner, which prunes the trial if its best intermediate result is worse than the median of intermediate results of previous trials at the same step.

The values in bold in Table 3 were the ones giving the best return after 100 000 training episodes.

We then trained the agent in the source environment for

1 000 000 episodes using the UDR algorithm with the parameters found through GridSearch.

4.4. Automatic Domain Randomization

We tested different configuration values for:

- *lower threshold* and *upper threshold*: values compared to the performance to see if a change in the lower or upper bound is needed
- *buffer size*: the number of episode rewards stored before an evaluation happens
- *step Δ* : how much the boundary is increased or decreased after the performance evaluation

We found that a buffer size of 25 with a lower threshold of 900 and an upper threshold of 1300 was a good compromise between good results and keeping the training time limited (1 000 000 episodes).

For Δ , we tested three different approaches:

- $\Delta = \Delta^0$
- $\Delta = \Delta^0 \cdot \frac{(p-t_H)}{100}$, where p is the performance and t_H is the upper threshold (when comparing the performance with the lower threshold t_L we have the companion formula $\Delta = \Delta^0 \cdot \frac{(-p+t_L)}{100}$)
- $\Delta = \Delta^0 \cdot \log_{10} \left(\frac{p-t_H}{10} \right)$ (when comparing the performance with the lower threshold we have the companion formula $\Delta = \Delta^0 \cdot \log_{10} \left(\frac{-p+t_L}{10} \right)$)

In the first approach, Δ is fixed throughout the training. In the other two approaches, instead, it is proportional to the difference between performance and threshold. In one case, we used a “simple” proportion, and in the other, we used a logarithm.

In particular, we found good results with the following configurations:

- $\Delta = 0.075$
- $\Delta = 0.011 \cdot \frac{(p-t_H)}{100}$
- $\Delta = 0.075 \cdot \log_{10} \left(\frac{p-t_H}{10} \right)$

5. Results

5.1. REINFORCE vs Actor-Critic

For 10 runs per algorithm, we computed the reward over 10 test episodes every 250 training episodes. We then averaged the reward over the 10 different runs. The resulting graph is shown in Figure 1.

As we can see, Actor-Critic performs better than the two REINFORCE implementations, reaching a mean reward of

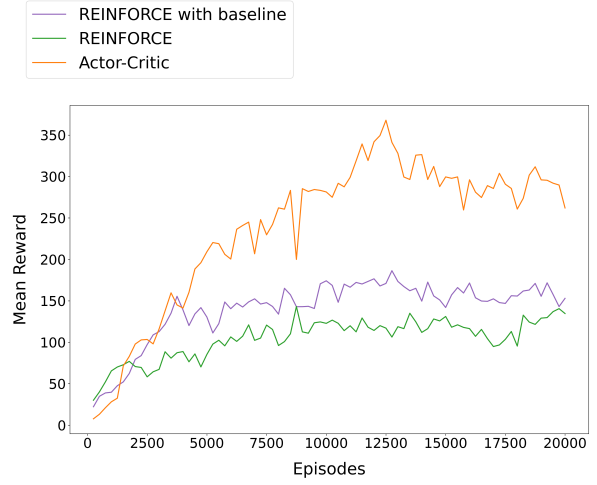


Figure 1. Comparison between REINFORCE and Actor-Critic

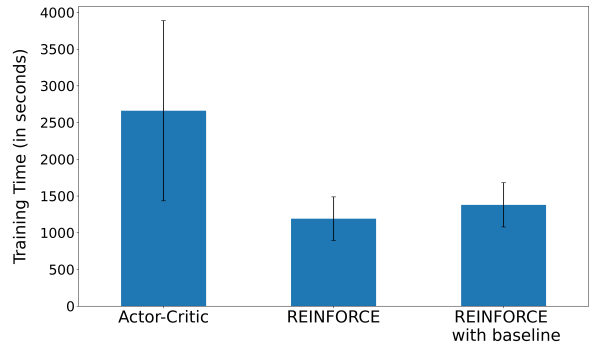


Figure 2. Comparison between REINFORCE and Actor-Critic

262 after 20 000 episodes. We can also see that using a baseline in REINFORCE gives better results than without.

In Figure 2, we can see that Actor-Critic has a significantly higher mean training time compared to both REINFORCE’s implementations. This notwithstanding, we should consider that Actor-Critic reaches higher results in far fewer episodes.

5.2. Proximal Policy Optimization

For every scenario (*Source* $\rightarrow$ *Source*, *Source* $\rightarrow$ *Target* and *Target* $\rightarrow$ *Target*), the agent was tested for 50 episodes after 1 000 000 training episodes. In Table 4, we can see their performances.

There’s a gap in performance between *Source* $\rightarrow$ *Target* and *Target* $\rightarrow$ *Target*. This was to be expected: the Sim-to-Real gap is caused by a discrepancy between the two domains. The same happens in this case: the agent trained

Training and evaluation environments	Return
Source $\rightarrow$ Source	1434
Source $\rightarrow$ Target	846
Target $\rightarrow$ Target	1767

Table 4. Search Space for PPO

in the source domain when tested in the target environment encounters different dynamics not experienced before. In our case, the “different dynamics not experienced before” are the shift of 1 kg in the torso’s mass.

Training directly on the target environment led to a higher return, but this is impractical in reality for the reasons explained in section 1.2.

The following sections try to reduce this gap between the lower bound (*Source $\rightarrow$ Target*) and upper bound (*Target $\rightarrow$ Target*).

5.3. Uniform Domain Randomization

We trained the agent in the source environment for 1 000 000 episodes using the UDR hyperparameters in Table 3. We tested the agent in the target environment for 50 episodes. The return was 1183, resulting in an improvement compared to the lower bound of PPO without UDR (Table 4).

As we’ve seen, UDR is an adequate starting point in bridging the performance gap caused by Sim-to-Real. However, there are two crucial drawbacks:

- the high effort needed in manual tuning
- the environment is randomized from the beginning of training, instead of a gradual inclusion of randomized parameters

Domain Randomization	Return
UDR	1183
AutoDR with fixed Δ	1463
AutoDR with proportional Δ	1564
AutoDR with log proportional Δ	1570

Table 5. Implementations of Domain Randomization

5.4. Automatic Domain Randomization

In Figure 3, we can see a comparison between UDR and AutoDR with a fixed Δ . The two agents were evaluated for 50 episodes every 20 000 training episodes. The return at the end of training is 1183 for UDR, as said in the previous paragraph, and 1463 for AutoDR. AutoDR seems to have higher peaks, but also lower lows compared to UDR. We can see the potential AutoDR has, but it probably takes more episodes to get to more stable returns.

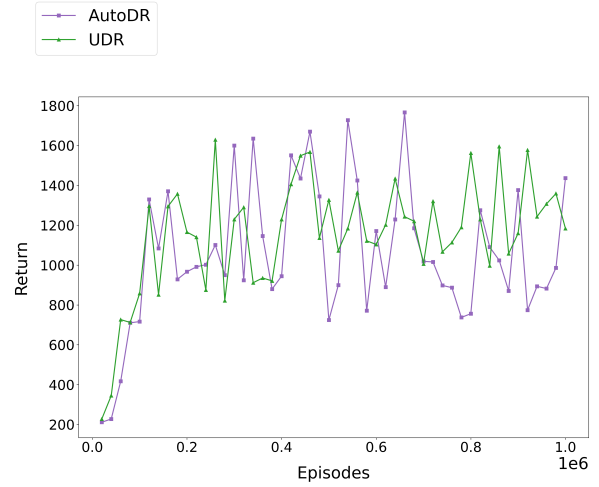


Figure 3. Comparison between AutoDR and UDR

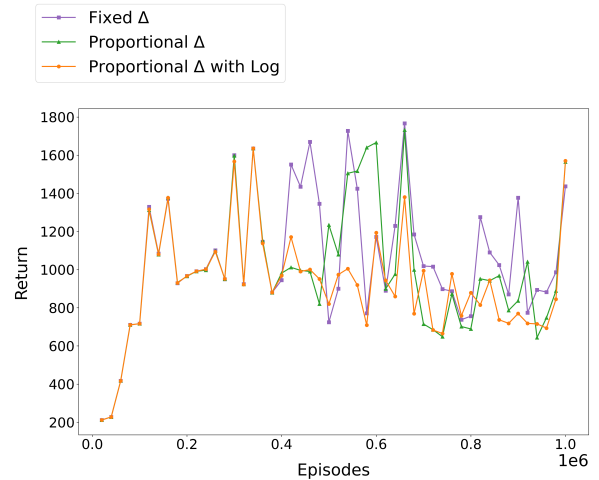


Figure 4. Comparison between the AutoDR implementations

We can now compare the results of the three different implementations of AutoDR. In Figure 4, we can see their returns plotted by evaluating for 50 episodes every 20 000 training episodes. In Table 5, we can see their returns after 1 000 000 training episodes.

AutoDR with a fixed Δ seems to have a better return throughout most of the training, but in the end, the two proportional approaches have a better result. An interesting extension of this project could be to continue the training for more episodes and see if this trend continues or the AutoDR with fixed Δ comebacks.

6. Discussion

When comparing REINFORCE and Actor-Critic in section 4.1, we trained the model 10 times to have a result that is not considerably affected by the intrinsic randomness of Reinforcement Learning algorithms. Instead, the `stable-baseline3` library offers ways to make computations deterministic. Thus, the results in sections 5.2, 5.3 and 5.4 are overall reproducible (with some limitations that the authors of the library are still trying to overcome). This was a necessary feature during our experiments: we needed to have the same results between tuning and training (otherwise, tuning would have been of no use), but also to make the various results comparable with each other.

In section 4.2, we trained and tested the agent in the target environment to tune the hyperparameters. This is not what would happen in a real application, in that case, the source environment (the only one easily available) would be used to tune the hyperparameters. We made the executive decision of using the target environment, in order to maximize the return for $Target \rightarrow Target$, such that we had to bridge an even bigger gap with Domain Randomization. With this choice made, we can see how easily Domain Randomization works even in the worst circumstances.

We expected a better return with AutoDR compared to UDR, and while the AutoDR algorithm performs better, it's still far from the $Target \rightarrow Target$ performance and the return varies drastically between one evaluation and the next. We can think of a few adjustments. A first solution could be implementing AutoDR for more episodes, with a bigger buffer size, and a smaller Δ . Hence, the changes in the bounds would happen more slowly, making it possible for the agent to learn better from it. A second solution, without drastically increasing the training time, could be changing how boundary sampling works. We had a relatively small buffer size, meaning that even with the randomness provided by the algorithm, the various buffers filled out one after the other. Thus, the bound changed in rapid succession, resulting in a more difficult scenario in just a few episodes.

7. Conclusion

In this paper, we observed the performance of some of the main Reinforcement Learning algorithms, showing how increasing the complexity of the algorithms can yield much better results. We then saw how even more complex algorithms like PPO are not robust enough to adapt to a slightly different environment. We tried to bridge this gap through one of the most known techniques to address the Sim-to-Real gap: Domain Randomization. The UDR algorithm provided the expected results: a performance increase, but with difficult tuning to find the right parameters. The ADR algorithm provided results worse than expected, but we think it's possible to get an improvement through a

few tweaks to our application of the algorithm.

References

- [1] Yegane Bagheri and Chiara Roberta Casale. Bridging the sim-to-real gap with domain randomization. <https://github.com/chiararcasale/Bridging-the-Sim-to-Real-Gap-with-Domain-Randomization/>, 2024. 1
- [2] Wenshuai Zhao, Jorge Peña Queralta, and Tomi Westerlund. Sim-to-real transfer in deep reinforcement learning for robotics: a survey. *arXiv preprint arXiv:2009.13303*, 2020. 1
- [3] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2018. 2
- [4] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world, 2017. 2, 3
- [5] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. Solving rubik's cube with a robot hand, 2019. 2, 3
- [6] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning series. The MIT Press, second edition, 2018. 3
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. 3
- [8] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. 4
- [9] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework, 2019. 4